

Offline Software Setup Guide

Step 1: Install Python

1. Open your browser and go to **python.org/downloads**.
2. Point at **Downloads** in the top menu. Under **Download for Windows**, find the line *"Or get the standalone installer for Python 3.14.x"* and click that **Python 3.14.x** button.
 - Don't pick **"Python install manager"**.
3. Open the downloaded file, python-3.14.x-amd64.exe.
4. On the first screen, **tick "Add python.exe to PATH"** at the bottom. This is the one step people most often miss.
5. Click **Install Now** and wait for it to finish.
6. On the last screen, if it offers **"Disable path length limit"**, click it. Then click **Close**.

Check it worked:

1. Press the **Windows key**, type cmd, and open **Command Prompt**.
2. Type python --version and press Enter.
3. You should see **Python 3.14.x**.

Why: The computer's processor only understands machine code, instructions in 0s and 1s. Our file clinic_app.py is just plain text: English-like lines such as db.execute("INSERT INTO patients ..."). You could open it in Notepad.

Python (the program python.exe) is an interpreter. It reads that text line by line, translates each line into instructions the processor can run, and runs them immediately. No Python, no translation, so the .py file is just text that does nothing.

Python also comes with a standard library: ready-made code that our app uses without us writing it. **tkinter** draws windows and buttons, **sqlite3** runs the database, **csv** writes Excel files, **zipfile** makes backups. Installing Python installs all of these.

"Add python.exe to PATH": Windows keeps a list of folders called PATH. When you type a command like python in Command Prompt, Windows searches those folders for python.exe. Ticking the box adds Python's folder to that list; otherwise, Windows says "not recognized".

Step 2: Install VS Code (the editor where you'll read and write code)

1. Go to **code.visualstudio.com**.
2. Click the blue **Download for Windows** button.
3. Open the downloaded file (VSCodeUserSetup-x64-...exe).
4. Accept the agreement and click **Next** through the screens.
5. On the "**Select Additional Tasks**" screen, tick:
 - **Add "Open with Code" action to Windows Explorer file context menu**
 - **Add "Open with Code" action to Windows Explorer directory context menu**
 - **Add to PATH** (usually already ticked)

The first two let you right-click any folder and open it in VS Code, which you'll use constantly.

6. Click **Install**, then **Finish**. VS Code opens.

Why: A .py file is a text file, so any text editor could write it. VS Code is a code editor: it shows line numbers, highlights code by type (commands, text, numbers in different colours), lets you search across the whole file, and has a built-in terminal (a Command Prompt inside the editor) to run the program without switching windows.

Step 3: Add the Python extension to VS Code

This teaches VS Code to understand Python: it colours the code, points out mistakes, and adds a ► Run button.

1. In VS Code, click the **Extensions** icon on the left edge (four small squares, one slightly apart). Or press **Ctrl+Shift+X**.
2. In the search box at the top, type **Python**.
3. Click the first result, **Python** by **Microsoft** (it has a blue tick for the verified publisher).
4. Click **Install**.
5. Wait a few seconds until the button changes to **Uninstall** or a gear icon. That means it's installed.

It may also install **Pylance** and **Python Debugger** alongside it. That's normal; they're part of the same package.

Why: VS Code itself is general-purpose. It supports dozens of languages, but knows none of them deeply out of the box. The extension adds three things for Python:

- Pylance, a language server: it reads your code as you type and underlines errors (a misspelled name, a missing bracket) before you even run it, and suggests completions.
- Debugger: lets you pause the program at a chosen line and inspect values, to find out why something goes wrong.
- Interpreter link: connects VS Code to the python.exe you installed, so the ► Run button knows which Python to use.

Step 4: Install Pillow (the image library)

1. Make sure the internet is on.
2. Open **Command Prompt** (Windows key → type cmd).
3. Type this and press Enter:
4. `pip install pillow`
5. Wait for **Successfully installed pillow-12.x.x**.

If it says 'pip' is not recognised, use `python -m pip install pillow` instead.

Why: The standard library can't create or edit images. **Pillow is a third-party library: code written by other developers**, published free, that adds image functions. Our app can use it to create a blank A4 image, paste the logo, and write text with Marathi fonts onto it. In the code it can appear as `from PIL import Image, ImageDraw, ImageFont`.

pip is Python's package installer. `pip install pillow` downloads Pillow from PyPI (the Python Package Index, the official online store of Python libraries) and puts it in Python's site-packages folder. From then on, any Python program on this PC can import it.

Step 5: Install DB Browser for SQLite

1. Go to sqlitebrowser.org → **Download**.
2. Under Windows, click **DB Browser for SQLite - Standard installer for 64-bit Windows**.
3. Open the downloaded .msi file and click **Next** through the screens.
4. On the **Shortcuts** screen, tick **Desktop** under "DB Browser (SQLite)".
5. Click **Install**, then **Finish**.

One rule to keep from day one: **never open the real clinic.db while Clinic Records is running**, and never edit patient data in it by hand. We'll only practise on test databases.

Why: Our app stores everything in one file, clinic.db. That's an **SQLite database**: tables with rows and columns, like Excel sheets, but stored in a format only programs can read. Double-click it and Windows won't know how to open it.

DB Browser is a free viewer and editor for SQLite files. It shows each table (patients, visits, medicines) as a grid, so you can see exactly what the app saved. It also lets you type **SQL commands**, the same language our code uses (SELECT, INSERT), and see the results instantly. That's the fastest way to learn how the database side works. The app itself doesn't need it; it's a tool for you to look inside.

SQLite needs no separate installation. It came with Python in Step 1.

What SQLite actually is: a database engine, the code that knows how to store tables in a file, find rows quickly and run SQL commands. It's written in C and is small enough to be built into other programs.

Python bundles it in two parts:

- **The engine:** a file called `sqlite3.dll`, in Python's DLLs folder.
- **The Python module:** `sqlite3`, part of the standard library. When our code says `import sqlite3`, this module connects Python to that engine.

Why that matters: many databases, like MySQL or PostgreSQL, are **servers**. They're separate programs you install, start, keep running in the background, and set up with usernames and passwords. Your program connects to them like a website connects to a server.

SQLite is **embedded**: no server, no separate program, no setup. The engine runs inside our app, and the whole database is **one ordinary file**, e.g. `clinic.db`. That's why backing up is just copying a file, and why moving to a new PC is just moving that file. It's the right choice for a single-clinic, offline app.

You can see it's already there. In Command Prompt, type:

```
python -c "import sqlite3; print(sqlite3.sqlite_version)"
```

It prints the engine's version, something like 3.50.x.

And, **DB Browser is optional**. The app never needs it. It's only a viewer so you can look inside .db files while learning. Install it if you'd like to see the tables, or skip it.

SQLite is a small database engine, built into Python, that stores all the data as tables in a single file. The difference matters in one way: SQLite isn't only Python's. Android phones, WhatsApp, Chrome and thousands of other programs use the same engine. Python just includes it ready to use.

Step 6: Make a practice folder and open it in VS Code

1. Open File Explorer and go to any folder let's say **Local Disk (D:)**.
2. Right-click an empty area → **New** → **Folder** → name it e.g. "**ClinicSoftware**".
3. Open **VS Code**.
4. Click **File** → **Open Folder...** → select **D:\ClinicSoftware** → **Select Folder**.
5. If it asks "**Do you trust the authors of the files in this folder?**", click **Yes, I trust the authors**. You made the folder, so it's safe, and without this VS Code runs in a restricted mode.

You'll see **ClinicSoftware** in the left sidebar, empty for now.

Why: VS Code works with a **folder as a project** (it calls it a *workspace*). Everything for one project lives in one folder: the .py files, the database file it creates, and later the build output. Opening the folder, rather than a single file, lets VS Code see all the files together in its sidebar, and its terminal starts **inside that folder**.

That last point matters: when the code says `sqlite3.connect("mini_clinic.db")`, with no full path, Python creates or opens the file in the **current working directory**, the folder the program was started from. Starting from our practice folder means the test database lands there, far away from the real clinic data in `Documents\ClinicRecords`.

Step 7: Tell VS Code which Python to use

1. In VS Code, press **Ctrl+Shift+P**. A command box opens at the top.
2. Type **Python: Select Interpreter** and press Enter.
3. From the list, pick the one that says **Python 3.14.x** with a path containing **Python314**, like:
~\AppData\Local\Programs\Python\Python314\python.exe
4. Check the bottom-right corner of VS Code: it should now show **3.14.x**.

If the list shows no 3.14, click **Enter interpreter path... → Find...** and browse to C:\Users\Satyajit\AppData\Local\Programs\Python\Python314\python.exe.

The bottom-right Python version only appears once a .py file is open, so if you don't see it yet, don't worry. We'll create the first file in the next step and check it then.

Click **Restricted Mode** at the bottom left screen of VSCode.

Click **Trust**, or **Trust Folder and Continue** if it asks that way.

The Restricted Mode label disappears.

Now press **Ctrl+Shift+P** again, type **Python: Select Interpreter**, and the Python commands will show. Pick **Python 3.14.x** (path containing Python314).

Why: PCs may have more like Pythons: 3.14 (from Step 1), 3.12 and Anaconda (both from 2024). Each is a separate installation with its **own** site-packages folder, so a library installed into one isn't visible to the others. Pillow went into **3.14** (you saw the path: Python314\Lib\site-packages). If VS Code ran our code with Anaconda's Python instead, import PIL could fail or load a different version. Choosing the **interpreter** fixes which python.exe the ► Run button uses for this folder.

Step 8: Test the whole setup with a small check program

1. In the left sidebar, point at **CLINICSOFTWARE** and click the **New File** icon (a page with a +).
2. Name it **check.py** and press Enter. It opens in the editor.
3. Paste this in:

```
import sys
import sqlite3
import tkinter
import PIL

print("Python version :", sys.version.split()[0])
print("Python location:", sys.executable)
print("SQLite engine :", sqlite3.sqlite_version)
print("tkinter      :", tkinter.TkVersion)
print("Pillow       :", PIL.__version__)
print("All tools ready!")
```

(If error- Click inside the code

Press **Ctrl+A** to select all.

Press **Shift+Tab** two or three times, until every line starts right at the left edge, with no spaces before import or print.)

4. Press **Ctrl+S** to save.
5. Click the ► **Run** button at the top right of the editor.

A **terminal** panel opens at the bottom and prints six lines, ending with **All tools ready!** The location line should contain **Python314**.

What the code does: the import lines load each library, so if any is missing, Python stops right there with an error naming it. Each print line displays one fact: the Python version, which python.exe is running, and each library's version.

Why: before writing the real app, we confirm every piece works together: VS Code runs the right Python, and that Python can load all four libraries our app needs (sqlite3, tkinter, Pillow, sys). If anything is missing, it fails here, in one clear line, rather than halfway through a bigger program.

Lines 1–4 imported libraries. A library is ready-made code someone else wrote. import loads it into our program so we can use its functions, instead of writing everything ourselves. Without an import, Python doesn't know those names: writing `sqlite3.connect(...)` alone would fail with "name 'sqlite3' is not defined".

Library	What it is	What our clinic app uses it for
sys	Information about Python itself and the computer it runs on	Here, to print Python's version and location. In the app, to check whether it's running on Windows.
sqlite3	Connects Python to the SQLite database engine	Creating tables, saving patients and visits, searching (INSERT, SELECT, UPDATE)
tkinter	Python's toolkit for windows and controls	Every window, text box, button, dropdown and list you see in the app
PIL (Pillow)	Image creation and editing	Drawing the prescription: blank A4 page, logo, text in Marathi and English

The first three came with Python (the **standard library**). Only Pillow was installed separately with pip, which is why we checked it. It's imported as PIL because it's a continuation of an older library called PIL (Python Imaging Library), and kept the name.

One more thing appeared in the sidebar: a **.vscode** folder. VS Code created it to remember this folder's settings, including the Python you chose in Step 7. Leave it there.

Step 9: Write Python Code

This code can be written by hand, or generated with the help of AI tools.

The whole software is one Python file, e.g. clinic_app.py. When it runs, it opens a window where the clinic can register patients, record visit history, track follow-ups, attach lab reports and create prescriptions. All data is stored on the same computer in one database file, so no internet is needed.

Libraries used:

- **tkinter** (built into Python): windows, forms, buttons and lists
- **sqlite3** (built into Python): the database that stores patients, visits and medicines in a single file, clinic.db
- **Pillow** (installed with pip): draws the prescription image with the clinic logo and letterhead
- **json, csv, zipfile, shutil** (built into Python): the question list, Excel export, backups and file copies

How the code is organised:

1. **Database:** creates the tables and saves, searches, updates and deletes records using SQL commands (CREATE TABLE, INSERT, SELECT, UPDATE, DELETE).
2. **Actions:** functions that run when a button is clicked, for example reading the form, checking the entries and saving them.
3. **Windows:** the screens the user sees: the main patient list, the visit form, follow-ups and settings.
4. **Start:** opens the main window and waits for the user's clicks and typing.

The data is saved in Documents\ClinicRecords, separate from the program, so the software can be updated without affecting patient records.

Step 10: Package the mini app as an .exe with a Desktop shortcut

Why: right now the app only runs from VS Code or with `python mini_clinic.py`. A doctor won't do that. **PyInstaller** bundles three things into a single .exe: a copy of the Python interpreter, the libraries your code imports (tkinter, sqlite3), and your script. The .exe then runs by double-clicking, **even on a PC without Python**.

1. The VS Code terminal starts with **(base)**, which is Anaconda switching itself on. In that state, typing `python` may run **Anaconda's Python 3.13**, and PyInstaller would get installed into the wrong Python. So, in the VS Code terminal:

```
conda deactivate
python --version
```

(base) disappears from the prompt, and the version should say **Python 3.14.7**.

2. **Install PyInstaller** (internet on). In terminal, type

```
python -m pip install pyinstaller
```

3. **Build the .exe**

```
python -m PyInstaller --onefile --windowed --name MiniClinic mini_clinic.py
```

It takes about a minute and prints many lines. It's finished when the prompt returns, with a last line mentioning **Build complete**.

What the options mean:

- `--onefile`: everything packed into a single .exe
- `--windowed`: no black command window behind the app
- `--name MiniClinic`: the .exe is called MiniClinic.exe instead of mini_clinic.exe

4. **Find the result**

Three new things appear in the sidebar:

- **dist\MiniClinic.exe**: your program, about 10–12 MB
- **build**: PyInstaller's scratch work (you can ignore or delete it)
- **MiniClinic.spec**: the recipe PyInstaller used, reusable for future builds

5. Test it

Double-click the new Desktop shortcut. The first start takes 2–3 seconds, because a one-file .exe unpacks itself into a temporary folder each time it starts. The grey line at the bottom should show **Data file: D:\ClinicSoftware\dist\mini_clinic.db**. That's the whole point of the code change: started from the Desktop, it still finds the right database.